

Agency Blog SaaS: Demo

Deep Dive

A scripted, fabricated product tour of a private production system

Read this before anything else

This document describes **agency-blog-saas-demo**: a small static web page that **imitates** a product. It is not the product.

The real system, **agency-blog-saas**, is a production Django application serving paying clients of a real agency. Its source is private and stays private, because it is the agency's live product rather than a codebase that could be scrubbed and published. This demo exists so that the product's screens can be clicked through by someone who will never be given the real repository.

Almost everything the demo displays is **fabricated**: the credit balance, the client, the client's updates, the generation process itself. Nothing is computed. Nothing is sent anywhere. The one exception is the blog library, which reproduces real, already-public articles from a real client's public website, and this document is careful to say which is which every time it matters.

Contents

1	What this project is	2
1.1	The one-paragraph version	2
1.2	Why a demo repository exists at all	2
1.3	What is honestly demonstrated, and what is not	3
2	Running it	3
3	The layout of the repository	4
4	The three tiers of truth	5
4.1	Tier 3 is the unusual choice	5
4.2	The labelling machinery, concretely	6
5	Architecture	6
5.1	How the page assembles itself	7
5.2	The data model	7
5.3	The claimed figures, and what could be checked	9
6	app.js: every behaviour on the page	9
6.1	The state	9
6.2	The tab machine	10
6.3	Autofill	10
6.4	The scripted generation	11
6.5	The demo card's life cycle	12
6.6	The article reader	13
6.7	Escaping, and the one place it is skipped	14
6.8	Wiring	15
7	The self-contained guarantee	15
8	styles.css: a borrowed design system	16
9	Licensing	17
10	Everything that was verified, and how	17
11	The honest summary	18
11.1	What is genuinely good here	18
11.2	What does not hold up	19
11.3	What this demo cannot demonstrate, and should not be asked to	19

1 What this project is

1.1 The one-paragraph version

`agency-blog-saas-demo` is a single web page, written in plain HTML, CSS and JavaScript, with no build step, no framework, no server of its own and no network calls at runtime. It presents four tabs that mirror the main screens of a private Django product: an AI Blog Generator, a Blog Library, Subscription Packages, and Client Updates, plus a fifth hidden panel that reads a full article in place. Clicking **Generate post** runs a pre-written timeline of status messages, subtracts a number from a variable, and copies a post that was already stored in the page into the library with a countdown on it. That is the whole program.

The distinction that governs this entire document

There are two separate things with similar names.

The real system (`agency-blog-saas`) is a production Django SaaS behind `digitalise.agency`. It has a database, user accounts, real billing, real AI generation, PDF export, and real clients. **You cannot see it**, and neither can a hiring panel.

This demo (`agency-blog-saas-demo`) is roughly 1,200 lines of static files, of which the large majority is copied article text. It has no database, no accounts, no billing, no AI, and no clients. **It only knows how to draw pictures of those things.**

Every claim in this document is about the demo unless it explicitly says otherwise. Where the demo makes a claim about the real system, this document treats that claim as a claim, not as a verified fact, because the code here cannot verify it.

1.2 Why a demo repository exists at all

A portfolio has a structural problem: the most substantial work is often the work that cannot be shown. An agency's live product is commercially sensitive in a way that no amount of secret-scrubbing fixes, because the sensitive thing is not a password in a config file, it is the product. Deleting the credentials from a live product's source and publishing the rest still publishes the product.

There are three usual responses, and it is worth naming why this project chose the third.

1. **Publish nothing.** Honest, but the reader has only a written claim to go on.
2. **Publish screenshots.** Better, but a screenshot cannot be clicked, and a reader cannot tell a screenshot of a working product from a screenshot of a mockup.
3. **Publish a fabricated, clearly-labelled reimplementations of the screens.** The reader gets something to click. The cost is that what they are clicking is a puppet, and the whole design problem becomes making sure nobody mistakes the puppet for the real thing.

This project is option three, and most of the interesting decisions in it are about managing the cost of that choice rather than about the code.

Jargon: Static site

A website made of files that a server hands over exactly as they are stored: HTML, CSS, images, JavaScript. Nothing is generated per visitor. The opposite is a *dynamic* site, where a program on the server (Django, for example) builds a different page for each request by reading a database. The real system is dynamic. This demo is static, which is precisely why it can pretend to be dynamic only by scripting the pretence in the browser.

1.3 What is honestly demonstrated, and what is not

This matters enough to be a table rather than a paragraph.

Product capability	What the demo shows	What that proves
AI blog generation	A button, a form, three status messages on timers, and a pre-stored post appearing.	That a generation screen exists and roughly what it feels like. <i>Nothing</i> about prompting, model choice, queueing, or output quality.
Credits and metering	A number in the corner that decreases by 100.	The concept. Not the ledger, not the refund-on-failure logic, not the race conditions that make metering hard.
Subscriptions	Three price cards and a redemption box that always says it is a demo.	The tier structure and price points, which are real and public. Not payment, not provisioning, not renewal.
Per-client editor roles	Nothing. There is no login and no notion of a user.	Nothing. This capability is named in the demo's own prose but is absent from the demo.
PDF export	Nothing. There is no export button.	Nothing. Also named in the prose, also absent.
Client updates	A three-item list for an openly invented company.	The screen's shape only.
Published output quality	Fifteen genuine, full-length articles from a real client's live blog.	This is the one place where something real is on display. See section 4.

Table 1: Capability by capability: the demo is a picture of a product, and two of the capabilities it advertises in its own hero text are not pictured at all.

The demo advertises two features it does not contain

The page's own opening paragraph tells the visitor that the real system does "AI blog generation, credits and subscriptions, per-client editor roles, and PDF export". The first two have tabs. **Per-client editor roles and PDF export have no representation anywhere in the demo:** no login, no role switcher, no export control, no mention on any tab.

This is not dishonest, because the sentence is explicitly describing the real system rather than the demo, and the very next sentence says the page is a scripted tour. But it is a real gap between the advertised feature list and the tour: a visitor who reads the list and then clicks all four tabs looking for roles and export will not find them, and will not be told why. Either the tour should cover them or the hero should not lead with them.

2 Running it

```

1 # 1. Open the file directly.
2 xdg-open index.html          # or double-click it
3
4 # 2. Or serve the folder with any static server.
5 python3 -m http.server 8000  # then visit http://localhost:8000

```

Listing 1: There is no setup. These are the only two ways to run it.

Both work identically, because the page never makes a request that a `file://` origin would block. There is no `npm install`, no compiler, no environment file and no secrets, and this is a deliberate property rather than an accident: see section 7.

`netlify.toml` is the entire deployment configuration:

```
1 [build]
2   publish = "."
```

Listing 2: `netlify.toml`, in full

Jargon: Netlify, and what `publish = "."` means

Netlify is a hosting service that watches a git repository and serves its contents as a website. Most projects need a *build command* first, turning source into deployable files. This project has no build, so the configuration says only “publish the folder as it is”. Two lines of deployment configuration is a direct consequence of the no-build-step decision.

3 The layout of the repository

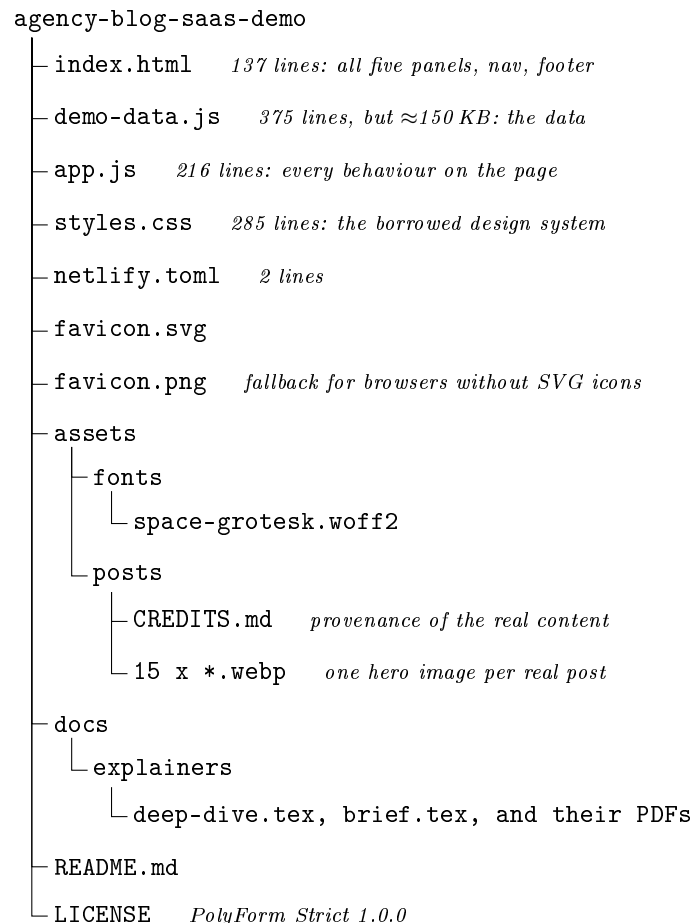


Figure 1: The whole project. Four files carry the entire program; the 2.5 MB of weight is almost all images and one font.

The file count is small enough that the architecture is really just an answer to one question: what goes in which of the four files? The answer is a clean and conventional split, and it is worth stating because it is one of the things a reader can actually judge here.

- `index.html` holds *structure only*: five `<section>` panels, each mostly empty, waiting to be filled.
- `demo-data.js` holds *data only*, and defines three global constants.
- `app.js` holds *behaviour only*, and reads those constants.
- `styles.css` holds *appearance only*.

The split is real but not absolute

Three of the panels in `index.html` carry inline `style="..."` attributes (`color:var(--ink-soft);font-size:14px` and similar) rather than classes, so a little appearance has leaked into the structure file. It is cosmetic and it is four attributes, but the separation the project otherwise keeps carefully is not perfectly kept.

4 The three tiers of truth

This is the most important idea in the project, and it is not a coding idea.

The data on the page is not uniformly fake. It comes in three tiers with genuinely different provenance, and the design of the page is largely an attempt to keep a visitor from confusing them.

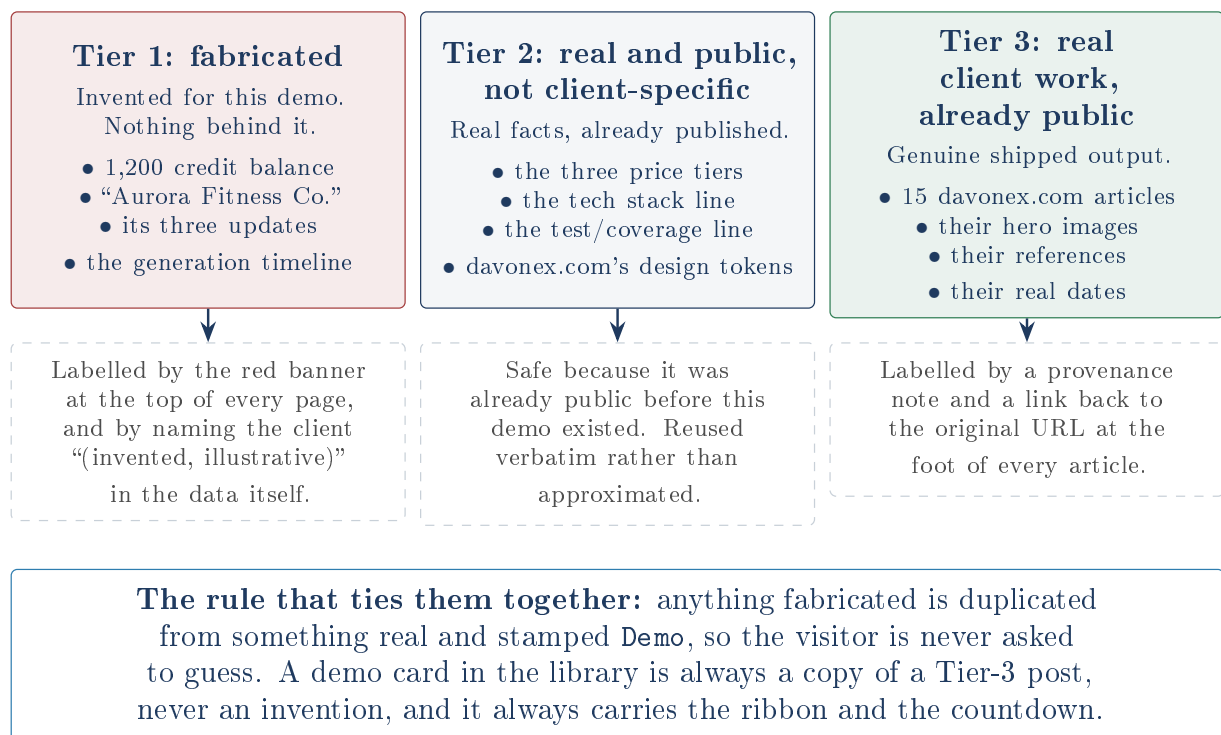


Figure 2: Three provenances on one page, and the labelling that keeps them apart.

4.1 Tier 3 is the unusual choice

Most fabricated demos invent their content library too: a page of lorem-ipsum blog posts with plausible titles. This one does not. It ships all fifteen real, live articles from davax.com/blog, an actual client, with their real titles, real publication dates, real read times, real hero images, real full article bodies (tables, headings, inline citations and all) and their real reference lists. Verified by counting them in the source: 15 posts, 15 unique slugs, 15 unique titles, 177 reference entries in total, and roughly 154KB of article HTML.

The project's `README.md` argues that this is a *stronger* honesty position than inventing a library, and the argument is sound: a reader can see genuine shipped work and a working interaction in one place, with a link back to the original on every article so any claim is checkable in one click.

But Tier 3 comes from a different private system than the one being demoed

`assets/posts/CREDITS.md` states that `davonex.com` is “an actual configured client of the real blog-posting-pipeline production system”. `blog-posting-pipeline` is a *different* private project from `agency-blog-saas`; both exist in this portfolio and both are private.

The page's own prose is looser. In the article reader and in the `README`, `davonex.com` is called “an actual configured client of the real production pipeline”, without saying which pipeline. A visitor reading that sentence inside a demo titled “Agency Blog SaaS” will very reasonably conclude that these fifteen posts were produced by Agency Blog SaaS. **The one file that names a system says otherwise.**

This is worth resolving before an interview, because it is the kind of thing that gets asked. Two readings are possible and the code cannot settle it: either the two systems share the client and the wording is merely imprecise, or the library is showcasing another project's output inside this project's demo. Whichever is true, the honest framing that the rest of the project works so hard for is undercut by the one ambiguous noun.

4.2 The labelling machinery, concretely

Four separate mechanisms in the code exist only to keep the tiers legible:

1. **A permanent banner.** `index.html` opens with a fixed red strip: Demo mode: fabricated product tour. No live API calls happen here; nothing shown is a real client's data. It is the first element in the body and it never goes away.
2. **Self-labelling data.** The invented client's name is literally stored as “Aurora Fitness Co. (invented, illustrative client)”. The label is part of the value, so it cannot be lost by a template that forgets to render a flag.
3. **The ribbon and countdown.** Any generated card wears a ribbon reading Demo, duplicate, and removes in 14:59 (joined by middle dots), ticking down live.
4. **The provenance note.** Every opened article ends with a paragraph stating the content is real, published, and stored locally, followed by a link to the original `davonex.com` URL. Verified in a real browser: the link resolves to `https://davonex.com/blog/<slug>/` for the post that is open.

The banner claim “nothing shown is a real client's data” is false

The banner says: *nothing shown is a real client's data*. But the Blog Library is fifteen real posts from a real client, and the demo's own `README`, `CREDITS` and article provenance notes all say so proudly.

The intended meaning is clearly “no private client data”, and everything shown is genuinely public. But as written, the page's most prominent honesty statement contradicts the page's most interesting honest feature. The footer has the same problem: “It contains no real client data”. A careful reader who notices this is being handed a reason to distrust every other label on the page, which is the exact opposite of what the banner is for.

This is cheap to fix and worth fixing: “no private client data; the Blog Library is a real client's already-public content, labelled as such” would be both accurate and a stronger claim.

5 Architecture

5.1 How the page assembles itself

There is no framework, so the load sequence is worth tracing exactly once, because everything else follows from it.

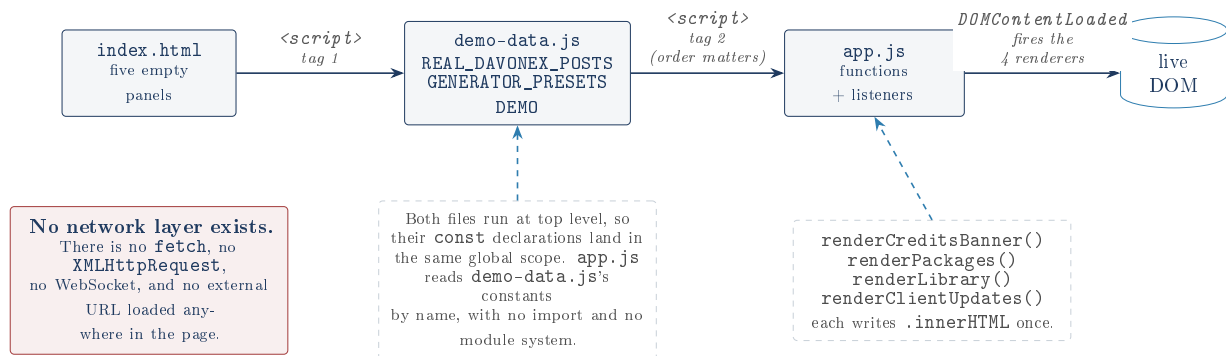


Figure 3: The complete architecture. Two script tags in a fixed order, three global constants, four render functions, one DOM.

Jargon: The DOM, and innerHTML

The DOM (Document Object Model) is the browser's live, in-memory tree of the page. HTML is the text you wrote; the DOM is the object the browser built from it and actually draws. JavaScript changes the page by changing the DOM.

`element.innerHTML = "<p>hi</p>"` is the blunt instrument for that: it throws away everything inside `element` and rebuilds it by parsing a string of HTML. It is simple and fast to write. Its two costs matter later in this document: it destroys and recreates the elements every time (section 6.5), and it will execute whatever markup the string happens to contain (section 6.7).

Jargon: Global scope, and why script order is load-bearing

A browser runs `<script>` tags in the order they appear. A `const` declared at the top level of a plain (non-module) script is visible to every script that runs after it.

So `app.js`'s very first executable line, `let creditsBalance = DEMO.credits;`, works only because `demo-data.js` appeared first in `index.html` and left `DEMO` lying in the global scope. Swap the two `<script>` lines and the page dies instantly with `DEMO is not defined`. This is a real coupling, invisible in either file, recorded only by the order of two lines in the HTML. It is completely standard for a page this size, and it is exactly what module systems were invented to make explicit.

5.2 The data model

`demo-data.js` defines three globals and nothing else.

```

1  const REAL_DAVONEX_POSTS = [
2    {
3      slug:      "track-ai-search-sales-e-commerce-attribution-guide-2026",
4      title:     "Track AI Search Sales: E-Commerce Attribution Guide (2026)",
5      excerpt:   "Learn how to track AI search sales for e-commerce. ...",
6      by:        "By Davonex",
7      date:      "Updated 6 Jul 2026",
8      read_time: "5 min read",
9      image:     "assets/posts/track-ai-search-sales-...-2026.webp",
10     body_html: "<p>To track real e-commerce sales from AI search, ...</p>...",
11     references: [
12       { title: "A Practical Framework for ...", url: "https://...", date: "2025-12-04" },
13       // ... 10 more

```



```

14   ]
15   },
16   // ... 14 more posts
17 ];

```

Listing 3: The shape of one entry in `REAL_DAVONEX_POSTS`. The `body_html` field is elided here; in the real file it is a single string of up to 20 KB of article markup.

Jargon: Slug

A short, lowercase, hyphenated, URL-safe version of a title. `Track AI Search Sales: E-Commerce Attribution Guide (2026)` becomes `track-ai-search-sales-e-commerce-attribution-guide-2026`. Here the slug does double duty: it is the identity of a post inside the demo (every lookup is `REAL_DAVONEX_POSTS.find(p => p.slug === slug)`) *and* it is the real path component on the real site, which is what lets the provenance link be built by string concatenation rather than stored.

The second constant is derived rather than written, and that derivation is the single most deliberate design decision in the codebase:

```

1  const GENERATOR_PRESETS = REAL_DAVONEX_POSTS.map(p => ({
2    slug: p.slug,
3    topic: p.title,
4    description: p.excerpt,
5    keywords: p.title.toLowerCase().replace(/[:]/g, "")
6               .split(" ").filter(w => w.length > 3).slice(0, 6).join(" "),
7  }));

```

Listing 4: `GENERATOR_PRESETS` is computed from the posts, not authored

Why derive the presets instead of writing them

A demo generator has an honesty trap in it. If “Autofill” writes one thing into the form and “Generate” produces something unrelated, the demo is quietly lying about the most basic property a generator has: that the output follows from the input.

By computing each preset *from* the post it will produce, the project makes the link structural rather than clerical. Preset *i* carries post *i*’s slug. Autofill records that slug. Generate looks it up. The same input always yields the same output because they are the same object, and no future edit can drift them apart, because there is nothing to keep in sync.

The README states that this mirrors the real pipeline’s actual deterministic-per-brief behaviour. That claim is about the private system and **cannot be checked from this repository**; treat it as the owner’s assertion, not as something this demo demonstrates.

The third constant holds the fabricated tour data:

```

1  const DEMO = {
2    credits: 1200,
3    packages: [
4      { name: "Basic",      price: "$9.99/mo",  credits: "500 credits",  note: "..."},
5      { name: "Pro",       price: "$24.99/mo",  credits: "1,500 credits", note: "...",
6        featured: true },
7      { name: "Enterprise", price: "$79.99/mo",  credits: "5,000 credits", note: "..."}
8    ],
9    client: {
10     name: "Aurora Fitness Co. (invented, illustrative client)",
11     updates: [ { date: "2026-06-01", text: "Published 4 blog posts this month, ..." },
12                { date: "2026-05-15", text: "Organic traffic up 12% month over month ..." },
13                { date: "2026-05-01", text: "Completed keyword research refresh ..." } ]
14   },

```

```

15   facts: {
16     tests:    "132/132 tests passing",
17     coverage: "56% coverage",
18     stack:    "Django 4.2, PostgreSQL 18, Docker, Gunicorn, nginx"
19   }
20 };

```

Listing 5: DEMO, in full

DEMO.facts is dead code, and its contents are duplicated by hand

Verified by searching the whole repository: `DEMO.facts` is **never read**. Not by `app.js`, not by `index.html`. It is defined and abandoned.

Worse than merely dead: the same three facts are hard-coded as literal text in the footer of `index.html`, which prints “Built with Django 4.2, PostgreSQL 18, Docker, Gunicorn, and nginx” and “132/132 tests passing, 56% coverage, per the real project’s README”. So the project has two copies of the same figures, one of which is wired to nothing. Change the data object and the page does not move; change the page and the data object silently lies.

The fix is one line in each direction and it should be done. As it stands this is a small but genuine defect in a project whose entire premise is that its claims are traceable.

5.3 The claimed figures, and what could be checked

The footer’s “132/132 tests passing, 56% coverage” is careful in one respect: it says *per the real project’s README*, attributing the number rather than asserting it directly.

Verified by cross-reference, not by this repository

These numbers are not checkable from inside this demo, which has no tests of its own and no access to the real system. Cross-referencing the private `agency-blog-saas` project’s own README confirms that it does state 132 tests and 56% line coverage against a 50% gate, and that it records them as locally verified.

So the demo is quoting its source accurately. What the demo cannot do, and what this document will not pretend it does, is *verify* them. A reader of the demo has the owner’s word, one step removed. That is the honest status of that footer line, and it is worth saying out loud in an interview before someone else asks.

6 app.js: every behaviour on the page

216 lines, no dependencies. This section covers all of it.

6.1 The state

The entire application state is five module-level variables.

```

1  let creditsBalance = DEMO.credits; // 1200, decremented by 100 per generate
2  let libraryDemo = null;           // { slug, addedAt, expiresAt, tickHandle,
3                                     //   timeoutHandle } or null
4  let currentPresetSlug = null;      // set by autofill, read by generate
5  let readerReturnTab = "library";   // where the Back button goes
6  const LIBRARY_LIFETIME_MS = 15 * 60 * 1000; // 15 minutes

```

Listing 6: The whole of the demo’s memory

Note what is *not* here: no array of generated posts. `libraryDemo` is a single slot, not a list. Generating twice replaces rather than appends. This is a deliberate constraint that keeps the library from filling up with duplicates, and it is enforced in `addToLibrary`.

6.2 The tab machine

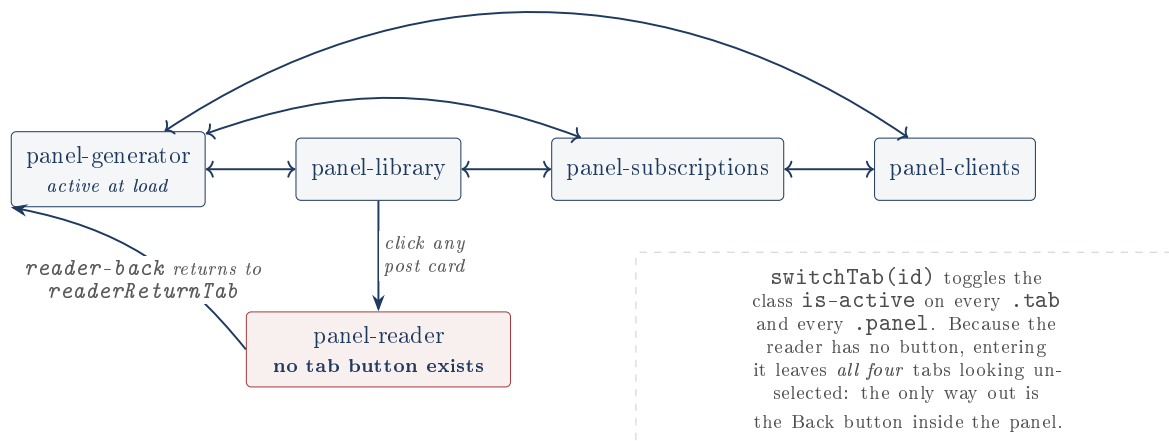


Figure 4: Five panels, four buttons. The reader is a panel that the tab bar does not know about.

```

1 function switchTab(tabId) {
2   document.querySelectorAll(".tab")
3     .forEach(t => t.classList.toggle("is-active", t.dataset.tab === tabId));
4   document.querySelectorAll(".panel")
5     .forEach(p => p.classList.toggle("is-active", p.id === `panel-${tabId}`));
6 }

```

Listing 7: The entire router

Two lines, and it is the right two lines for this problem. `classList.toggle(name, condition)` adds the class when the condition is true and removes it when false, so a single pass over every tab and every panel puts the whole UI into a consistent state with no bookkeeping.

No URL, no history, no deep link

The tab is not in the address bar. Refreshing always returns to the generator; the browser Back button leaves the page entirely rather than leaving the reader; and no article can be linked to. For a four-tab tour this is a defensible simplification, and adding `history.pushState` would be perhaps fifteen lines. But it does mean an interviewer cannot be sent a link to a specific screen, which for a demo whose purpose is to be shown to people is a slightly awkward limitation.

6.3 Autofill

```

1 function autofillRandom() {
2   const preset = GENERATOR_PRESETS[Math.floor(Math.random() * GENERATOR_PRESETS.length)];
3   currentPresetSlug = preset.slug;
4   document.getElementById("topic-input").value = preset.topic;
5   document.getElementById("desc-input").value = preset.description;
6   document.getElementById("keywords-input").value = preset.keywords;
7   document.getElementById("autofill-note").innerHTML =
8     `Filled with a real davonex.com post: <strong>${escapeHtml(preset.topic)}</strong>. ...`;
9 }

```

Listing 8: autofillRandom

`Math.random()` returns a number in $[0, 1)$; multiplying by the length and flooring gives a uniformly chosen index. The chosen slug is stashed in `currentPresetSlug`, which is the only channel between this function and the generator.

The derived keywords are visibly poor, and they are on screen

The keyword derivation is a title, lowercased, stripped of only () and :, split on spaces, filtered to words longer than three characters, and truncated to six. Running it over all fifteen real titles produces, among others:

```
1 google, best, e-commerce, 2026      <- from "Google Ads vs AEO: Best E-Commerce ROI
   in 2026"
2 what, answer, engine, optimization, e-commerce, 2026?
3 chatgpt, recommend, your, e-commerce, store, 2026
4 paid, e-commerce, lowering, 2026     <- from "SEO vs Paid Ads for E-Commerce:
   Lowering CAC in 2026"
```

Three concrete failures, all visible to any visitor who clicks Autofill:

- **The length filter deletes the subject.** “Ads”, “SEO”, “AEO”, “CAC”, “ROI” and “vs” are all four characters or fewer, so the Google Ads post’s keywords contain neither “ads” nor “AEO”, and the CAC post’s contain neither “SEO” nor “CAC”. The keywords omit precisely the terms the article is about.
- **Stopwords survive.** “what”, “your”, “best” and “guide” are longer than three characters, so they stay. No real keyword field would contain “your”.
- **The punctuation strip is incomplete.** It removes () and : but not ?, so one preset ships the literal keyword 2026?. Confirmed on screen in a real browser.

This matters more than a normal cosmetic bug, because this demo’s pitch is that its generator screen is a faithful picture of a real product. A keyword field reading `what, answer, engine, optimization, e-commerce, 2026?` is not a picture of a real product’s keyword field; it is a picture of a naive string split. Since the presets are derived rather than authored, the cheapest honest fix is a small stopword list and a hand-written keyword line per post, which is fifteen short strings.

6.4 The scripted generation

This is the centrepiece interaction, and it is four timers.

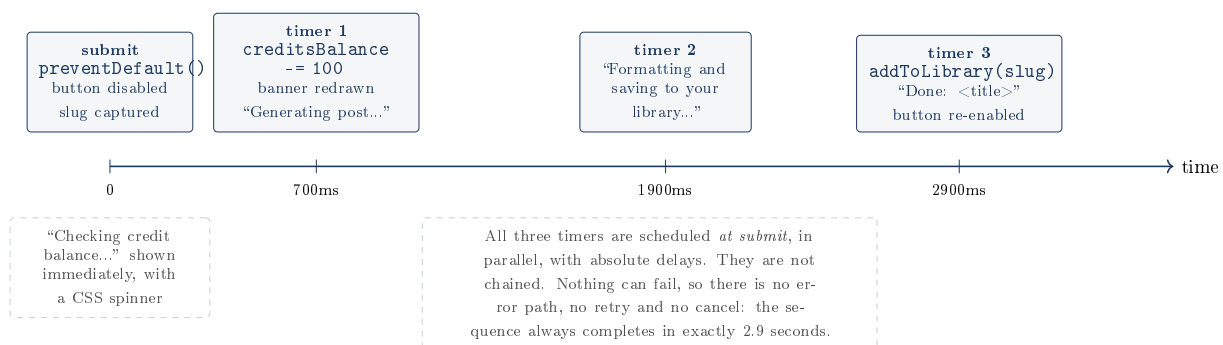


Figure 5: `runGenerator`: a fixed 2.9-second choreography of a process that does not happen.

Jargon: `setTimeout` and `preventDefault`

`setTimeout(fn, ms)` asks the browser to call `fn` once, after at least `ms` milliseconds, and returns immediately. It does not pause anything.

`e.preventDefault()` on a form’s `submit` event stops the browser’s built-in behaviour, which would be to send the form to a server and reload the page. Since there is no server, this one line is what makes the whole demo possible: the page stays put and JavaScript handles the event instead.

Generating without autofilling gives you someone else's post

```
const slug = currentPresetSlug || GENERATOR_PRESETS[0].slug;
```

If a visitor ignores the Autofill button, types their own topic, and hits Generate, then `currentPresetSlug` is still `null` and the fallback fires. Verified in a real browser: typing `My Own Unrelated Topic` and generating produces the status line `Done: Track AI Search Sales: E-Commerce Attribution Guide (2026) added to your Blog Library`.

The form is fully editable, none of the fields are `required` or `readonly`, and nothing tells the visitor that their input is being discarded. So the most natural first interaction on the page, typing something into the topic box, silently produces an unrelated article and asserts it as the result. That is the demo doing exactly the thing the preset-derivation design was built to prevent, one branch away from all that care.

The honest options are to disable Generate until Autofill has been clicked, or to make the inputs read-only with a note, or simply to say in the status line that the demo can only produce its fifteen stored posts. Any of the three costs a few lines. The current fallback is the one place where this demo says something untrue without labelling it.

Credits go negative

`creditsBalance -= 100` has no floor and no check. The starting balance is 1,200, so the thirteenth generation takes the display to `-100`, and it keeps going. Verified in a browser by driving the balance down: the banner will happily render `-50`.

In the real system this is the exact place where the interesting logic lives (refuse, refund on failure, race with a concurrent request). Here it is one subtraction. A visitor who clicks the button thirteen times sees a negative credit balance on a page that is supposed to be showing them how a metered product behaves. A single `if (creditsBalance < 100)` guard with an “out of credits” message would turn a bug into a demonstration of the feature.

6.5 The demo card's life cycle

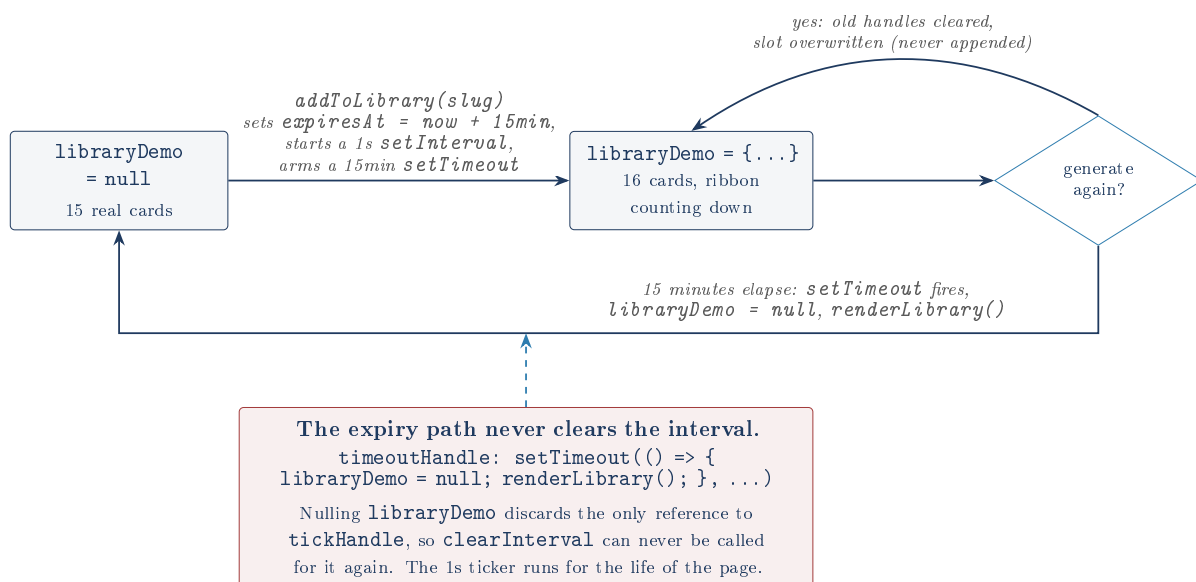


Figure 6: The single-slot life cycle, and the leak on the expiry edge.

```

1 function addToLibrary(slug) {
2   if (libraryDemo) { // replacing: clean up the old one
3     clearInterval(libraryDemo.tickHandle);
  
```

```

4   clearTimeout(libraryDemo.timeoutHandle);
5 }
6   const addedAt = Date.now();
7   const expiresAt = addedAt + LIBRARY_LIFETIME_MS;
8   libraryDemo = {
9     slug, addedAt, expiresAt,
10    tickHandle: setInterval(renderLibrary, 1000),
11    timeoutHandle: setTimeout(() => { libraryDemo = null; renderLibrary(); },
12                                LIBRARY_LIFETIME_MS)
13  };
14  renderLibrary();
15 }

```

Listing 9: addToLibrary, in full

The replace path is careful and correct: it clears both handles before overwriting the slot. The expiry path is not.

Verified leak: the ticker outlives the card, forever

Driven in a real headless Chrome. A `MutationObserver` was attached to `#library-list` to count rebuilds, then the expiry callback was executed exactly as `app.js` writes it.

```

1 DOM rebuilds of the 16-card grid in 3s while the demo card is alive: 4
2 DOM rebuilds in 3s AFTER the demo card expired:                    4  <-- still
   ticking

```

The interval keeps firing at 1 Hz forever, and each fire runs `renderLibrary()`, which rebuilds the *entire fifteen-card grid* via `innerHTML`: fifteen `<article>` elements, fifteen `<img>` tags, all destroyed and re-parsed, once a second, permanently, for a demo card that no longer exists. The handle is unreachable, so nothing can ever stop it short of a page reload.

The fix is one line: capture the handle in a local variable, or call `clearInterval(libraryDemo.tickHandle)` inside the timeout callback before nulling the slot.

Note the second-order cost too. Even *before* expiry, this design rebuilds fifteen image elements every second purely to update the `mm:ss` in one ribbon. Updating a single text node instead of re-rendering the grid would be both correct and cheaper. This is the clearest “I would do it differently” answer the codebase offers, and it is worth having ready.

6.6 The article reader

`openReader(slug, isDemo)` builds an entire article as one `innerHTML` string: hero image, an optional demo flag, title, byline, the stored `body_html` verbatim, a references list, and the provenance note with its link back to the original.

One small nicety before it renders:

```

1 readerReturnTab = document.querySelector(".panel.is-active")?.id.replace("panel-", "")
2                 || "library";

```

The reader remembers which panel it was opened from so Back returns there, defaulting to the library. Verified: Back does return to `panel-library`. In practice only the library has clickable cards, so the generality is unused, but it costs one line and it is correct.

Jargon: Optional chaining, `?.`

`a?.b` evaluates to `undefined` instead of throwing if `a` is `null` or `undefined`. Here it guards the case where no panel carries `is-active`, which then falls through to the `|| "library"` default. It is a defensive habit; the case cannot actually arise, because a panel is always active.

Every inline citation link in every article is dead

The real articles are full of citation markers: `<sup class="citation"><a href="#ref-1">[1]</a></sup>`. They are meant to jump to the matching entry in the references list.

But `openReader` renders the references as a plain `<ol>` of `<li>` elements with **no id attributes**, and searching all fifteen stored bodies confirms that no `id="ref-N"` target exists anywhere in the data either. So every one of those anchors points at an element that does not exist.

Measured in a real browser on one opened article: **21 citation links, 21 of them dead**. Clicking any of them does nothing at all.

This is the price of copying rendered HTML out of a site and re-hosting it in a different template: the markup came across, the anchor targets it depended on did not. It is a one-line fix (`<li id="ref-#{i+1}">` with an index from `map`), and it is worth doing precisely because these citations are the part of the demo that shows real, sourced work. A panel member who clicks a citation and gets nothing has just found the seam.

6.7 Escaping, and the one place it is skipped

```
1 function escapeHtml(str) {
2   return String(str).replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
3 }
```

Listing 10: The project's single security primitive

Jargon: Why escaping exists (XSS)

When you build HTML by pasting a string into a template, the browser cannot tell your markup from the pasted string's markup. If the string contains `<script>`, the browser runs it. This is cross-site scripting (XSS).

Escaping neutralises the characters that start markup: `<` becomes `<`, which the browser draws as a less-than sign rather than treating as the start of a tag. Escaping `&` first is essential and correctly ordered here; doing it last would double-escape the entities the earlier replacements just introduced.

The function is applied consistently to every title, excerpt, date, price and update text. Two deliberate exceptions and one apparent oversight:

- `post.body_html` is injected raw. This is intentional and unavoidable: it *is* HTML, and escaping it would print the tags as text.
- `post.image` is interpolated into a `src` unescaped. It is a local path from a static file, so nothing can reach it.
- `r.url` is interpolated into an `href` unescaped, in the references list.

The escaping is theatre, and it is worth knowing that

Every string `escapeHtml` touches is a hard-coded literal in `demo-data.js`. There is no user input anywhere on this page that reaches the DOM: the three form fields are read into `currentPresetSlug` logic and the redeem box is only tested for emptiness. Its value is never rendered.

So `escapeHtml` defends against nothing that can happen here. The unescaped `href` is equally harmless for the same reason: an attacker would have to edit the repository to exploit it, at which point they can do anything anyway.

This is a good habit applied to a codebase that does not need it, which is the right way round.

But it should be described accurately: the demo is safe because it has *no untrusted input*, not because it escapes carefully. The two are different arguments and only the first one is doing any work.

6.8 Wiring

```

1 document.addEventListener("DOMContentLoaded", () => {
2   renderCreditsBanner(); renderPackages(); renderLibrary(); renderClientUpdates();
3
4   document.querySelectorAll(".tab")
5     .forEach(t => t.addEventListener("click", () => switchTab(t.dataset.tab)));
6   document.getElementById("generator-form").addEventListener("submit", runGenerator);
7   document.getElementById("autofill-btn").addEventListener("click", autofillRandom);
8
9   document.getElementById("library-list").addEventListener("click", (e) => {
10     const card = e.target.closest("[data-open-post]");
11     if (!card) return;
12     openReader(card.dataset.openPost, card.dataset.isDemo === "1");
13   });
14   document.getElementById("reader-back").addEventListener("click", () =>
15     switchTab(readerReturnTab));
16   // ... redeem handler
17 });

```

Listing 11: DOMContentLoaded: the only entry point

Event delegation: the one non-obvious technique in the file

The library’s cards are destroyed and rebuilt every second while a countdown runs. A listener attached to a card would die with the card.

So no listener is attached to any card. One listener sits on the container, `#library-list`, which is never replaced. When a click arrives, `e.target` is whatever exact element was hit (possibly the image, or the title text), and `e.target.closest("[data-open-post]")` walks *up* the tree to find the nearest ancestor carrying that attribute, which is the card. If there is none, the click was on empty space and is ignored.

This is called event delegation. It works because events bubble up the DOM tree, and it is the correct answer to the “my listeners disappear when I re-render” problem. The card’s identity travels in `data-open-post` (the slug) and `data-is-demo`, read back via `dataset`, so the handler needs no closure over anything.

7 The self-contained guarantee

The demo makes a strong, testable promise, repeated in the banner, the footer, the README, and the first comment line of both JavaScript files: **no network calls**.

Why this is a real design constraint and not a slogan

An honesty claim that depends on a remote server is not an honesty claim. If the demo fetched `davonex.com`’s posts live, then: the page would break when the client’s site changed or went down; the demo would be reporting on a live system it does not control; and “no live API calls” would be false. Every article body, every hero image and the font itself were therefore fetched once (dated 2026-07-07 in `CREDITS.md`) and committed, which is why the repository is 2.5 MB for 1,200 lines of code.

The README is explicit that this is the constraint it refused to break, and names the cost: the fifteen-post snapshot will drift as the real client publishes more, and re-syncing was skipped rather than adding “the one exception that would break it”. That is a proportionate trade, honestly stated.

Verified, not assumed

The page was loaded in a real headless Chrome from a `file://` origin with every request intercepted and logged.

Result: 0 requests to any non-local origin. No HTTP, no fetch, no beacon, no analytics, no font CDN, no hotlinked images. The page also produced **zero JavaScript errors** across the full interaction: load, autofill, generate, open an article, go back, generate again, and expire the demo card.

The claim is true. This is the one headline claim the demo makes that this document can confirm from the outside rather than take on trust.

The font is part of that: `assets/fonts/space-grotesk.woff2` is committed and declared with a local `@font-face` rather than pulled from Google Fonts, which would have been a network call and a third-party request on every load.

8 styles.css: a borrowed design system

285 lines, opening with a comment that is itself a piece of the project’s honesty discipline:

```
1 Palette and component patterns below match the real, live public design
2 system read directly off davonex.com's own CSS (:root tokens, .card/.btn/
3 .eyebrow components), per the owner's direction to style this demo after
4 davonex.com rather than digitalise.agency's own indigo palette. Colors,
5 radii, and shadows are copied 1:1; nothing here is invented. This is public
6 visual design language (what every visitor's browser already renders), not
7 any site's private server code, auth, or billing logic, which stays out of
8 this repo entirely.
```

Listing 12: The header comment of styles.css, quoted

The tokens themselves are CSS custom properties:

```
1 :root {
2   --bg: #FAFBFE;           --paper: #FFFFFF;
3   --blue: #136BEE;         --blue-deep: #0C4497;   --navy: #062757;
4   --ink: #08152B;          --ink-soft: #46597A;
5   --line: rgba(6, 39, 87, .12);
6   --r-lg: 26px;            --r-md: 16px;           --r-sm: 10px;
7   --shadow-blue: 0 18px 40px rgba(19, 107, 238, .28);
8   --wrap: 960px;
9 }
```

Listing 13: A representative slice of the :root block

Jargon: CSS custom properties (variables)

`--blue: #136BEE` declares a named value on an element; `var(--blue)` reads it, and it cascades to all descendants. Declaring them on `:root` (the `<html>` element) makes them global. The point is that the palette lives in one block rather than being scattered as literal hex codes across 285 lines, so a token can be re-pointed once.

The rest of the file is unremarkable modern CSS, and readable: a fixed banner, a sticky blurred nav, a card with a hover lift and a top accent bar, arrow-animated buttons, a keyframed spinner,

a grid for the library, and an article template mirroring the real site’s post layout. There is no preprocessor and no framework, so what is in the file is what the browser gets.

The provenance argument is stated but not sourced

The comment says the values were “read directly off `davonex.com`’s own CSS” and copied 1:1. That is a specific, checkable claim, and it cannot be checked from this repository: there is no record of which stylesheet, at what date, and the demo makes no live request that could confirm it. `CREDITS.md` carefully dates the article snapshot (2026-07-07) but says nothing about the CSS.

The reasoning offered (that a public site’s rendered visual language is not private code) is reasonable, and the comment is admirably specific about the owner’s direction. But the project’s own standard elsewhere is to date and source its borrowings, and here it does not. Adding a line to `CREDITS.md` would close the gap.

9 Licensing

The project ships PolyForm Strict License 1.0.0, copyright 2026 Salman Adnan. This is a *source-available* licence, not an open-source one: it permits reading and evaluating the code, and forbids use, modification and redistribution.

Why a strict licence is the right call here

The repository contains a real client’s copyrighted article text and images, reproduced with permission from the owner’s own agency relationship rather than under any open licence (`CREDITS.md` says so explicitly). An MIT licence on this repository would purport to grant strangers rights to fifteen articles that are not the licensor’s to give away that freely. PolyForm Strict is the licence that matches what the repository actually is: a thing to be looked at, not a thing to be taken.

10 Everything that was verified, and how

The brief for this document requires that claims be run rather than assumed wherever that is cheap. It was cheap here. The page was driven in a real headless Chrome; the data file was evaluated in Node.

Claim	Result	Method
No network calls at runtime	True	0 non-local requests intercepted
No JavaScript errors across the full tour	True	0 pageerror, 0 console errors
15 real posts, all present at load	True	15 <code>.post-card</code> elements
15 unique slugs, 15 unique titles	True	Set comparison in Node
All 15 hero images exist on disk	True	<code>fs.existsSync</code> per post
Generate adds exactly one card	True	15 to 16, label (16)
Credits go 1,200 to 1,100 per generate	True	read from the DOM
Autofilled post is the generated post	True	titles compared, equal
Countdown starts at 15 minutes	True	ribbon read: <code>removes in 14:59</code>
Provenance link points at the open post	True	href read from the DOM
Back returns to the previous panel	True	<code>panel-library</code>
Article renders with tables and refs	True	13 paragraphs, 1 table, 11 refs
Citation links resolve	False	21 links, 21 dead
Typed topic is respected	False	unrelated post returned
Credits have a floor	False	rendered <code>-50</code>
Ticker stops when the card expires	False	4 rebuilds per 3s after expiry
<code>DEMO.facts</code> is used	False	no reference in the repository
132/132 tests, 56% coverage	Not verifiable here	matches the private project's README; the demo has no tests of its own
Real pipeline is deterministic per brief	Not verifiable here	claim about the private system
CSS tokens copied 1:1 from davonex.com	Not verifiable here	no dated source recorded

Table 2: Twelve claims confirmed, five defects confirmed, three claims that this repository cannot settle either way.

11 The honest summary

11.1 What is genuinely good here

- **The self-contained guarantee holds.** It is the demo's loudest claim and it survives measurement, at zero non-local requests. Most demos that promise this leak a font or an analytics beacon; this one does not.
- **Deriving the presets from the posts** is a real design decision with a real argument behind it, and it is the sort of thing worth being asked about.
- **The three-tier labelling** is more thought than most demos get, and shipping a real client's public work next to labelled fakes is a genuinely stronger position than inventing a library.
- **Event delegation on a re-rendering grid** is the correct technique, applied for the correct reason.
- **The code is small, clean and legible.** Four files, one job each, 216 lines of behaviour, no dependencies, no build. For what this needs to do, that is right.

11.2 What does not hold up

1. **The banner’s central claim is false as written** (“nothing shown is a real client’s data”), and it contradicts the project’s own best feature.
2. **The no-autofill path returns an unrelated post and calls it the result**, which is the one unlabelled untruth on the page.
3. **Every citation link in every article is dead**: 21 of 21 on the post measured.
4. **The interval leaks forever** and rebuilds fifteen images per second for the life of the page after a card expires.
5. **Credits go negative**, in a demo whose subject is metering.
6. **DEMO.facts is dead**, and its figures are duplicated by hand in the footer.
7. **The keyword derivation drops the subject of the article** and keeps the stopwords, on screen, in the flagship interaction.
8. **Two advertised capabilities** (per-client editor roles, PDF export) are not in the tour at all.
9. **The davonex provenance names a different private system** in CREDITS.md than the demo’s prose implies.

None of these is hard to fix. Items 1, 2, 3 and 5 are each a handful of lines, and they are the four that a careful visitor is most likely to trip over.

11.3 What this demo cannot demonstrate, and should not be asked to

It is worth being blunt, because the gap between the demo and the product is where an interview will go.

This repository contains no evidence about: whether the real generation works, how good its output is, how prompts are constructed, how credits are actually metered or refunded, how subscriptions are billed or provisioned, how per-client roles are enforced, how PDF export is done, how any of it performs, whether it is secure, whether it scales, or whether the tests it cites exist. It demonstrates that its author can build a small, clean, honest static page and think carefully about how to label a fiction. That is a real skill and it is on display here. It is not the same skill as building the Django system, and this demo is not evidence of that one.

The strongest defensible claim

“This is a puppet, I built the puppet carefully, and I labelled it as a puppet.” Everything above supports that sentence. Nothing above supports any sentence about the real product’s quality, and the demo is at its best when it does not try to.